

Tablas de base de datos consultadas directamente y propuesta de arquitectura para APIs de consulta

A continuación, se muestran las tablas de la base de datos que nuestra aplicación lee directamente y los datos que se emplean, los cuales requieren de una API para poder leer a través de los filtros necesarios para permitir un uso responsable de la información.

Los registros en estas tablas se utilizan para hacer validaciones a la hora de crear registros y para tomar decisiones respecto a como se va a manejar la creación, edición, consulta y eliminación de la información en el modelo de MonitoreApp creado por el equipo de U-devs. Es decir, se respeta la autonomía de la información del equipo original y no se reescribe nada directamente (Solo a través de las APIs proporcionadas hasta el momento).

Alguna de estas tablas ya cuentan con una API que permite obtener sus datos, pero recomendamos ampliarlas para incorporar todos los datos.

El listado de tablas es el siguiente:

c_accion_personal

- aceptar_restricciones_reversion
- accionGeneraSeparacion_id
- adenda_id
- ajuste_salario_id
- ausencia_id
- ausencia_transformada
- baja_id
- cambio_horario_id
- cambio_periodo_pago_id
- cantidad_horas
- categoriaEmpleado_id
- cliente_id
- comentarios
- consecutivo
- contratacion_id
- contrato_id
- coordinadoPor_id
- coordinador_id
- corpo_id
- document
- empleado_id
- empresa_id
- estado_aprobacion
- fecha_actualizacion
- fecha_aprobado_ec
- fecha_aprobado_jo
- fecha_fin
- fecha_fin_traslado
- fecha_insercion
- fecha_reversion
- fecha_sobrepuesto
- fecha_vence_justificar_ausencia
- fecha_vence_subir_adjunto
- horario_id
- id
- incapacidad_ccss_id
- incapacidad_ins_id
- libre_cubre_vacaciones_id
- licencia_id
- llegada_tardia_id
- mobile_upload
- monto_descontar_turnos
- motivo_reversion

- numero_hed
- numero_hem
- numero_hen
- operacion
- periodoPago_id
- permiso_con_goce_id
- permiso_sin_goce_id
- plaza_id
- preaviso_id
- puesto_id
- reemplazo_id
- reversible
- salario
- salario_base_diario
- salario_base_mensual
- salida_anticipada_id
- separacion_temp_id
- suspension_id
- tipoAccion_id
- tipoContratacion_id
- traslado_id
- traslado_temp_id
- usuario_actualizacion
- usuario_aprueba_ec
- usuario_aprueba_jo
- usuario_insercion
- usuario_reversion
- vacacionMes_id
- vacacion_disfrute_id
- vacacion_pago_id

c_cambio_guardia

- id

c_configuracion

- tiempo_gracia_marcar_salida

c_empleado

- categoriaEmpleado_id
- cedula
- codigo
- Email
- estado
- fecha_contratacion
- firma_manual
- id
- nombre
- password
- password_expires_at
- periodoPago_id
- primer_apellido
- segundo_apellido
- supervisor_id
- telefono
- tipoCedula
- tipoContratacion_id

c_empleado_datos_adjuntos_rrhh

- empleado_id
- fecha
- tipoDatoAdjunto_id

c_empleado_plaza

- division_id
- empleado_id
- plaza_id
- salario

c_horario

- id
- minutos_almuerzo

- tiene_almuerzo
- titulo

c_marca_dia

- cliente_id
- contrato_id
- corpo_id
- empleadoFijo_id
- empleadoReemplaza_id
- empresa_id
- fecha
- hora_entrada
- hora_entrada_digitada
- hora_fin
- hora_inicio
- hora_salida
- hora_salida_anticipada
- hora_salida_digitada
- horario_id
- horas_duracion
- id
- plaza_id
- puesto_id
- tipo_turno

c_tipo_accion

- nombre

e_estructura_articulo_corpo_puesto_entrega

- corpo_id
- fechaEntrega
- id
- marca
- modelo
- nomencladorArticuloCP_id
- puesto_id
- serie

e_estructura_articulo_corpo_puesto_plan

- articuloCP_id
- cantidad
- combo_id
- corpo_id
- id
- puesto_id

e_estructura_cliente

- empresa_id
- id
- nombre

e_estructura_combo_articulo_cp

- id
- nombre

e_estructura_contrato

- cliente_id
- division_id
- empresa_id
- id
- nombre
- nro_contrato

e_estructura_empresa

- codigo
- id
- nombre

e_estructura_plazas

- categoriaSalarial_id
- codigo_plaza
- id

- nombre
- nro_plaza
- puesto_id

e_estructura_puesto

- codigo
- comboArticulosCP_id
- coordenadas_gpslat
- coordenadas_gpslng
- id
- nombre
- sucursal_id
- tiene_relevo

e_estructura_sucursal

- contrato_id
- coordenadas_gpslat
- coordenadas_gpslng
- ejecutivoCuenta_id
- id
- nombre
- nro_sucursal

e_licencia

- empleado_id
- tipoLicencia_id
- vence

n_articulo_corpo_puesto

- id
- nombre

n_coordinador

- id
- nombre

n_division

- codigo
- id
- nombre

n_ejecutivo_cuenta

- id
- nombre

n_tipo_dato_adjunto_rrhh

- id
- nombre

n_tipo_licencia

- id
- nombre

pg_categoria_empleado

- codigo
- id
- nombre

pg_categoria_salarial

- categoriaEmpleado_id
- horas_extras_diurnas
- horas_extras_mixtas
- horas_extras_nocturnas
- id
- salario_dia
- salario_mes

Propuesta de arquitectura para APIs de consulta de información

La propuesta consiste en implementar una capa de APIs que permita consultar de forma estandarizada la información almacenada en las diferentes tablas de la base de datos MySQL.

El objetivo principal es disponer de mecanismos de consulta flexibles que permitan obtener tanto registros individuales como conjuntos de registros, utilizando diferentes criterios de filtrado y evitando la necesidad de desarrollar una lógica de consulta específica para cada posible criterio de búsqueda.

1. APIs específicas por tabla

Una primera alternativa consiste en crear un conjunto de endpoints asociados a cada tabla de la base de datos.

Para cada tabla se implementarían, como mínimo, dos tipos de endpoints:

1.1. Consulta de un registro individual

Un endpoint destinado a obtener un único registro de la tabla, permitiendo especificar dinámicamente la propiedad mediante la cual se realizará la búsqueda.

Por ejemplo, dependiendo de la tabla y de la necesidad de la consulta, el criterio podría corresponder a:

- id
- nombre
- puesto_id
- plaza_id
- cualquier otra columna disponible en el modelo.

La API debería recibir tanto el nombre de la propiedad/campo por el cual se desea realizar la búsqueda como el valor que debe utilizarse como criterio.

Conceptualmente, una solicitud podría representar algo similar a:

```
GET /api/{tabla}/one?field=id&value=123
```

o:

```
GET /api/{tabla}/one?field=puesto_id&value=45
```

También debería contemplarse la posibilidad de realizar búsquedas parciales sobre campos de texto mediante un operador equivalente a LIKE.

Por ejemplo, una búsqueda sobre el campo nombre podría solicitar todos aquellos registros cuyo nombre contenga determinado texto:

```
GET /api/{tabla}/one?field=nombre&operator=LIKE&value=Juan
```

Dependiendo de la definición del operador, esto podría representar conceptualmente una consulta como:

```
WHERE nombre LIKE '%Juan%'
```

No obstante, el uso de LIKE para una consulta individual debería devolver un único registro únicamente cuando el criterio garantice que existe un resultado único. Si pueden existir múltiples coincidencias, debería utilizarse el endpoint destinado a consultas múltiples.

La implementación deberá validar que el campo solicitado sea un campo permitido para evitar consultas arbitrarias o problemas de seguridad asociados con la construcción dinámica de

sentencias SQL.

1.2. Consulta de múltiples registros

El segundo endpoint estaría orientado a obtener múltiples registros de una determinada tabla.

Al igual que en la consulta individual, el criterio de filtrado deberá ser dinámico, permitiendo seleccionar el campo sobre el cual se desea realizar la consulta.

Por ejemplo:

```
GET /api/{tabla}/many?field=puesto_id&value=45
```

Además de búsquedas por igualdad, este endpoint debería contemplar diferentes operadores y tipos de filtros.

Entre ellos:

a. Búsqueda por un valor específico

Permitir obtener todos los registros cuyo campo coincida con un valor determinado.

```
puesto_id = 45
```

b. Búsqueda parcial mediante LIKE

Para campos de tipo texto, debería ser posible realizar búsquedas parciales utilizando un comportamiento equivalente al operador SQL LIKE.

Por ejemplo:

```
nombre LIKE '%Juan%'
```

Esto permitiría encontrar registros en los que el texto buscado aparezca en cualquier posición del campo.

También podría contemplarse la posibilidad de especificar diferentes modalidades de búsqueda, por ejemplo:

- LIKE '%Juan%' !' contiene "Juan"
- LIKE 'Juan%' !' comienza con "Juan"
- LIKE '%Juan' !' termina con "Juan"

La API debería definir claramente cuáles de estas modalidades estarán disponibles y cómo se especificará cada una mediante sus parámetros.

Este tipo de consulta estaría especialmente orientado a campos como:

- nombres
- apellidos
- descripciones
- códigos
- observaciones
- nombres de puestos
- nombres de plazas
- u otros campos de naturaleza textual.

c. Búsqueda utilizando una lista de valores

Permitir consultar registros cuyo valor se encuentre dentro de una lista determinada de valores. Esto sería equivalente conceptualmente a un operador SQL IN.

Por ejemplo:

```
puesto_id IN (10, 15, 20, 25)
```

Esto permitiría enviar múltiples identificadores en una única solicitud en lugar de ejecutar una petición independiente por cada elemento.

También debería contemplarse, cuando sea necesario, la operación inversa (NOT IN).

d. Comparaciones numéricas

Para campos numéricos debería ser posible utilizar operadores de comparación como:

- >
- <
- >=
- <=
- !=

Por ejemplo:

```
id > 100
```

O:

```
puesto_id != 45
```

e. Filtrado por rangos de fechas

Para campos de tipo DATE, DATETIME o equivalentes, se debería permitir especificar un rango temporal.

Por ejemplo:

```
fecha >= '2026-09-01'  
AND  
fecha <= '2026-09-10'
```

La API debería permitir establecer, según corresponda, una fecha inicial, una fecha final o ambas.

También sería conveniente definir claramente el comportamiento cuando los campos correspondan a DATETIME, particularmente respecto a horas, minutos y segundos, para evitar resultados inesperados en consultas que utilicen únicamente fechas.

f. Combinación de criterios

Cuando sea necesario, debería contemplarse la posibilidad de combinar varios filtros dentro de una misma consulta.

Por ejemplo:

```
puesto_id = 45  
AND  
estado = 1  
AND  
fecha BETWEEN '2026-09-01' AND '2026-09-10'
```

También podría combinarse una búsqueda parcial con otros criterios:

```
nombre LIKE '%Juan%'
AND
estado = 1
```

La implementación deberá establecer de forma explícita qué operadores y combinaciones estarán permitidos, evitando exponer directamente la capacidad de ejecutar SQL arbitrario desde los parámetros de la API.

2. API genérica de consultas

Como alternativa a la creación de endpoints específicos para cada tabla, se propone implementar una API genérica de consulta.

Esta alternativa podría resultar más conveniente considerando la cantidad de tablas existentes y la posibilidad de que posteriormente se incorporen nuevas tablas o nuevos criterios de búsqueda.

En este modelo, un único endpoint recibiría los parámetros necesarios para determinar:

- La tabla que se desea consultar.
- Si se requiere un único registro o múltiples registros.
- El campo o campos utilizados como criterio de búsqueda.
- El valor o valores utilizados para el filtro.
- El operador que se desea aplicar.
- El tipo de dato del campo, cuando sea necesario para determinar el tratamiento del valor.
- En caso de consultas por fecha, el rango correspondiente.
- Opcionalmente, otros parámetros relacionados con ordenamiento, paginación o límite de resultados.

Conceptualmente, podría manejarse mediante una estructura similar a:

```
GET /api/query
```

con parámetros que permitan especificar la consulta.

Por ejemplo, una consulta por igualdad:

```
table = empleados
field = puesto_id
operator = =
value = 45
```

Una consulta mediante LIKE:

```
table = empleados
field = nombre
operator = LIKE
value = Juan
```

Una consulta mediante una lista:

```
table = empleados
field = puesto_id
operator = IN
values = [10, 15, 20]
```

Y una consulta por rango de fechas:

```
table = actividades
field = fecha
operator = BETWEEN
from = 2026-09-01
to = 2026-09-10
```

La estructura exacta de los parámetros queda sujeta al diseño de la API y deberá definirse formalmente en la documentación técnica.

3. Operadores de consulta

Como parte del diseño de la API, sería conveniente definir un conjunto limitado y explícito de operadores soportados.

Como mínimo, podría contemplarse:

Operador	Descripción	Ejemplo conceptual
=	Coincidencia exacta	estado = 1
!=	Diferente de	estado != 1
LIKE	Coincidencia parcial de texto	nombre LIKE '%Juan%'
IN	El valor pertenece a una lista	id IN (1,2,3)
NOT IN	El valor no pertenece a una lista	id NOT IN (1,2,3)
>	Mayor que	id > 100
<	Menor que	id < 100
>=	Mayor o igual	id >= 100
<=	Menor o igual	id <= 100
BETWEEN	Dentro de un rango	fecha BETWEEN A AND B

No todos los operadores necesariamente deberían estar disponibles para todos los tipos de datos. Por ejemplo, LIKE debería estar restringido principalmente a campos de tipo texto, mientras que BETWEEN podría utilizarse para fechas o valores numéricos.

La API debería realizar la validación correspondiente antes de ejecutar la consulta.

4. Validación y seguridad

En caso de implementar una API genérica, es especialmente importante que el sistema no permita que el cliente envíe directamente fragmentos de SQL para ser ejecutados contra MySQL.

La API debería trabajar sobre una lista controlada de:

- tablas disponibles para consulta;
- columnas permitidas;
- operadores soportados;
- relaciones permitidas, en caso de contemplarse;
- tipos de datos;
- límites máximos de resultados.

De esta forma, el servidor podría recibir una solicitud como:

```
table = empleados
field = nombre
operator = LIKE
value = Juan
```

y encargarse internamente de interpretar el operador y construir la consulta correspondiente.

En ningún caso el valor recibido debería convertirse directamente en una sentencia SQL sin parametrización.

Esto permitiría mantener control sobre las consultas que pueden ejecutarse y reducir el riesgo de SQL Injection o de acceso no autorizado a tablas o columnas que no deberían estar expuestas mediante la API.

Adicionalmente, para las consultas LIKE, debería prestarse especial atención al uso de comodines (% y _) y definir si estos serán generados automáticamente por la API o si podrán ser especificados por el consumidor.

Por ejemplo, podría definirse que:

```
operator = LIKE
value = Juan
```

implique automáticamente:

```
LIKE '%Juan%'
```

en lugar de exigir al consumidor que envíe los comodines.

5. Paginación y volumen de información

Debido a que algunas tablas pueden contener un volumen considerable de registros, la API de consulta múltiple debería contemplar paginación.

Por ejemplo:

```
page = 1
limit = 100
```

o un mecanismo equivalente basado en offset/limit o paginación por cursor, dependiendo del comportamiento y volumen esperado de las consultas.

También sería conveniente establecer un límite máximo de registros retornados por solicitud para evitar que una consulta sin filtros o con filtros demasiado amplios genere respuestas excesivamente grandes y afecte el rendimiento del servidor o de la base de datos.

Esto es especialmente relevante para las consultas mediante LIKE, ya que una búsqueda parcial demasiado amplia podría coincidir con una cantidad elevada de registros.

Creemos que es correcto que el sistema pida una validación de token de acceso, pero veríamos conveniente que no hubiese restricciones acerca de lo que se puede acceder según el token de acceso (Es decir, que, si un registro pertenece a un usuario diferente a aquel que consulta, el sistema no frustre la búsqueda por temas de inconsistencia).

Recuerden que cualquier duda, pueden consultar y les responderemos tan pronto nos sea posible.

